COMPUTER ENGINEERING

QUEZON CITY

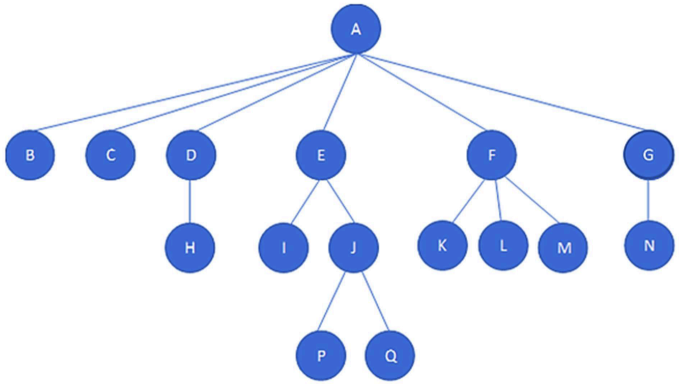
COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	September 18, 2026	**DATE SUBMITTED:**	September 24, 2026
NAME:	Silao, Johrel Louie T.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _9.1_ Tree ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Tree Diagram



Code

```cpp
#include <iostream>

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

int main() {
    Node* root = new Node(27);

    root->left = new Node(14);
    root->right = new Node(35);

    root->left->left = new Node(10);
    root->left->right = new Node(19);
    root->right->left = new Node(31);
    root->right->right = new Node(42);

    std::cout << "Creation of Binary Tree is Successful!" << std::endl;

    return 0;
}
```

Observation

This creates a binary tree using dynamically allocated node pointers which forms a balanced parent and child relationship at the root node.

Table 9-1: Creation of Tree

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2: Height and Depth of each Node

Observation:

- This table shows where depth increases as the element moves farther down the tree from the root node, while height represents the distance from the root node down to its deepest leaf node.

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Pre-Order	A B C D H E I J P Q F K L M G N
Post-Order	B C H D I P Q J E K L M F N G A
In-Order	B A C H D I E P J Q K F L M N G

Table 9-3: Traversal Strategies of Pre-Order, Post-Order, and In-Order

Observation:

- This traversal strategy shows the different kinds of operations for ordering sequences of the general tree nodes.

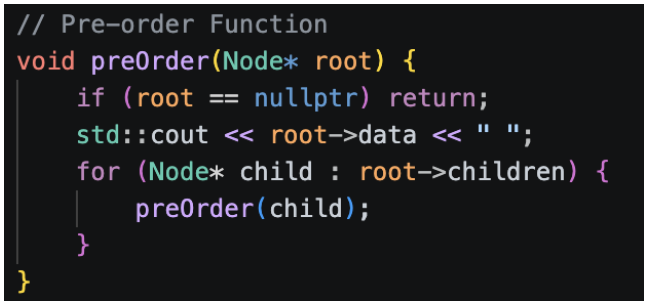
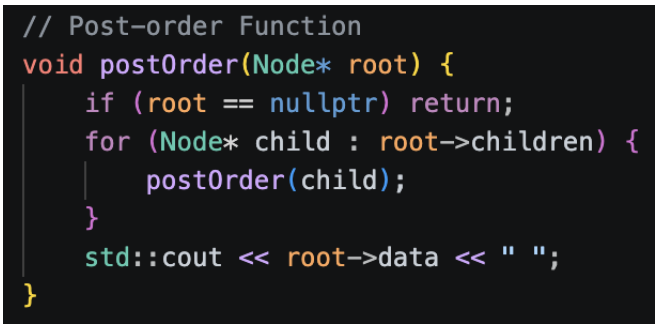
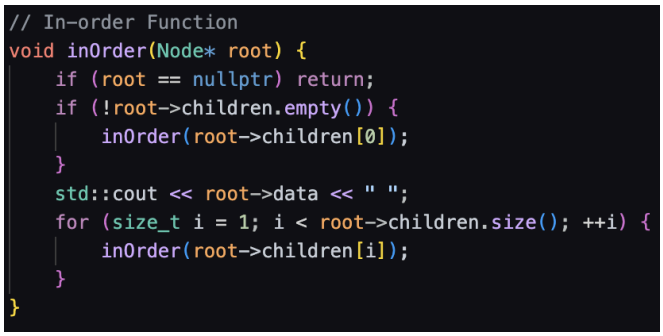
Pre-Order Code Screenshot	
Output	
Post-Order Code Screenshot	
Output	
In-Order Code Screenshot	
Output	
Observations	These functions iterate through each node's children vector which will display the expected output of the program.

Table 9-4: Creation of Pre-Order, Post-Order and In-Order Functions

Code Screenshot	

```cpp
// Task 3.3: findData
// Pre-Order helper search
bool searchPreOrder(Node* root, char key) {
    if (root == nullptr) return false;
    if (root->data == key) return true;
    for (Node* child : root->children) {
        if (searchPreOrder(child, key)) return true;
    }
    return false;
}

// Post-Order helper search
bool searchPostOrder(Node* root, char key) {
    if (root == nullptr) return false;
    for (Node* child : root->children) {
        if (searchPostOrder(child, key)) return true;
    }
    return (root->data == key);
}

// In-Order helper search
bool searchInOrder(Node* root, char key) {
    if (root == nullptr) return false;
    if (!root->children.empty()) {
        if (searchInOrder(root->children[0], key)) return true;
    }
    if (root->data == key) return true;
    for (size_t i = 1; i < root->children.size(); ++i) {
        if (searchInOrder(root->children[i], key)) return true;
    }
    return false;
}

void findData(Node* root, std::string CHOICE, char KEY) {
    bool found = false;
    if (CHOICE == "PRE") {
        found = searchPreOrder(root, KEY);
    } else if (CHOICE == "POST") {
        found = searchPostOrder(root, KEY);
    } else if (CHOICE == "IN") {
        found = searchInOrder(root, KEY);
    } else {
        std::cout << "Invalid traversal choice!" << std::endl;
        return;
    }

    if (found) {
        std::cout << KEY << " was found!" << std::endl;
    }
}

// Helper function
Node* findNode(Node* root, char key) {
    if (root == nullptr) return nullptr;
    if (root->data == key) return root;
    for (Node* child : root->children) {
        Node* res = findNode(child, key);
        if (res != nullptr) return res;
    }
    return nullptr;
}
```

Output	
Observations	This findData function will navigate or search through the tree structure that is based on the selected traversal method and displays a message when entering the character 'P'

Table 9-5: Creation of findData Function

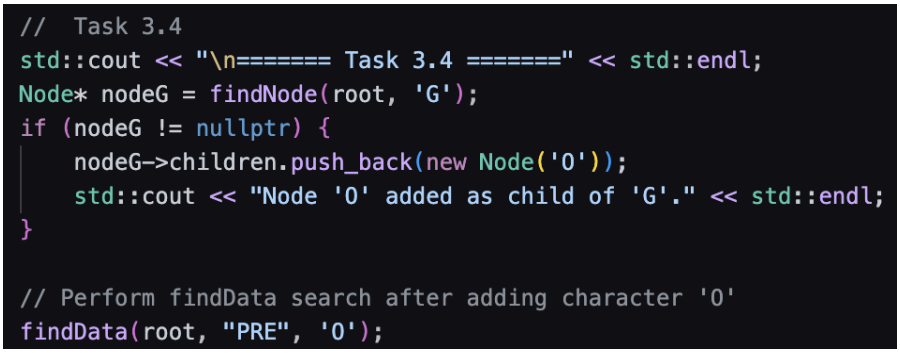
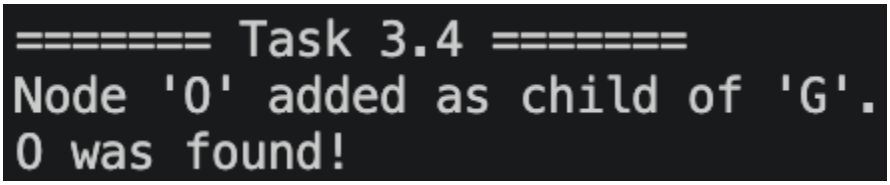
Code Screenshot	
Output	
Observations	This appends the node 'O' to node 'G' which will update the general tree's structure and calls the 'findData' function where it will verify that the newly added node is observable while traversing.

Table 9-6 Creation of New Leaf Node

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
#include <iostream>
#include "BinaryTree.h"


int main () {
    // Create the root node
    BinarySearchTree<int> tree(2);

    std::cout << "========== BINARY SEARCH TREE ==========" << std::endl;
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout << "Pre Order: ";
    tree.preOrder();
    std::cout << std::endl;

    std::cout << "In Order: ";
    tree.inOrder();
    std::cout << std::endl;

    std::cout << "Post Order: ";
    tree.postOrder();
    std::cout << std::endl;
    std::cout << "=======================================";

    return 0;
}
```

Figure 1: Code Snippet of Implementing a Binary Search Tree

Observations of Figure 1:

- Instantiates the root with '2' and sequentially inserts the remaining values which creates a dynamically binary search tree.

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

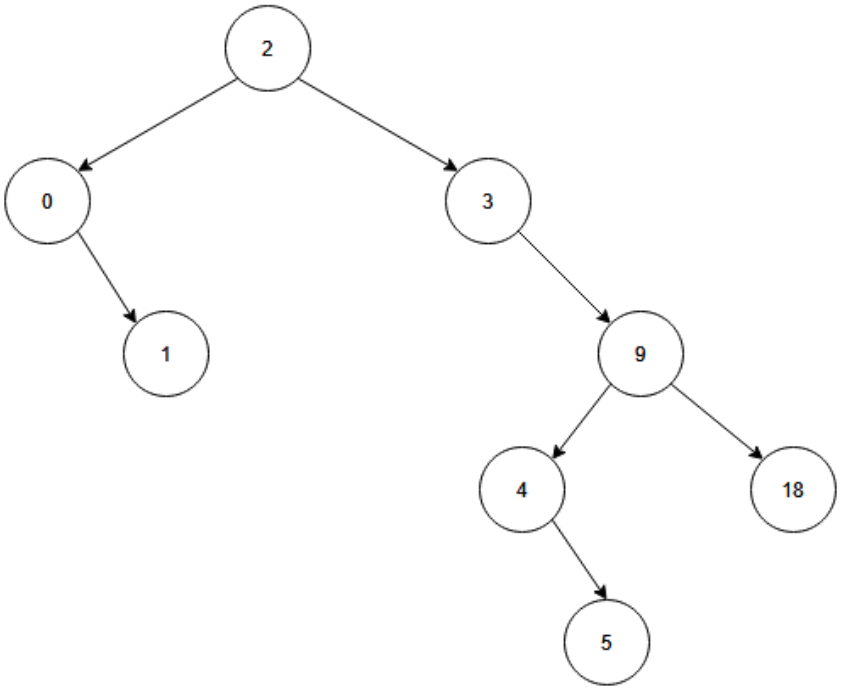


Figure 2: Tree Diagram

Observations of Figure 2:

- This provides a visual representation that illustrates how nodes are arranged relative to their parent keys.

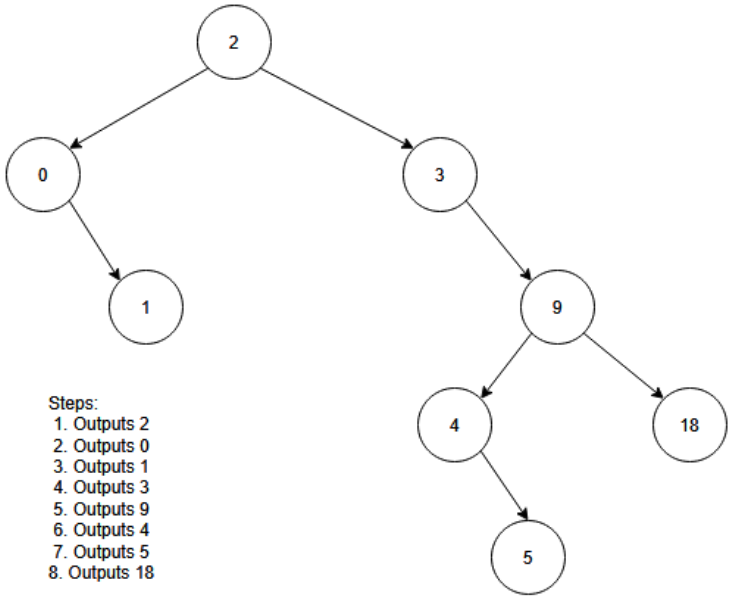


Figure 3: Pre-Order Traversal Diagram

Observations of Figure 3:

- This diagram demonstrates the root first traversal sequence that visits parent nodes before moving on to the left and right subtrees.

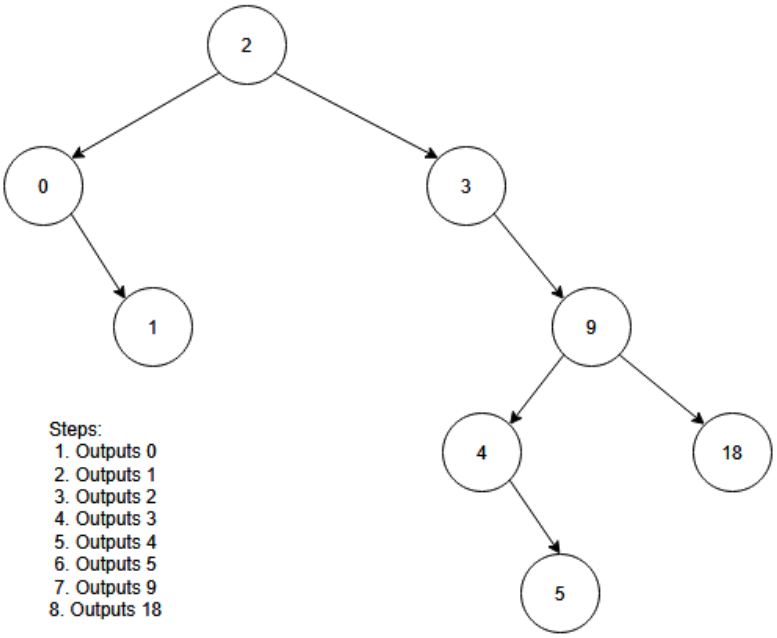


Figure 4: In-Order Traversal Diagram

Observations of Figure 4:

- This diagram shows how visiting left subtrees, parents and then right sub trees which will display an ascending order sequence.

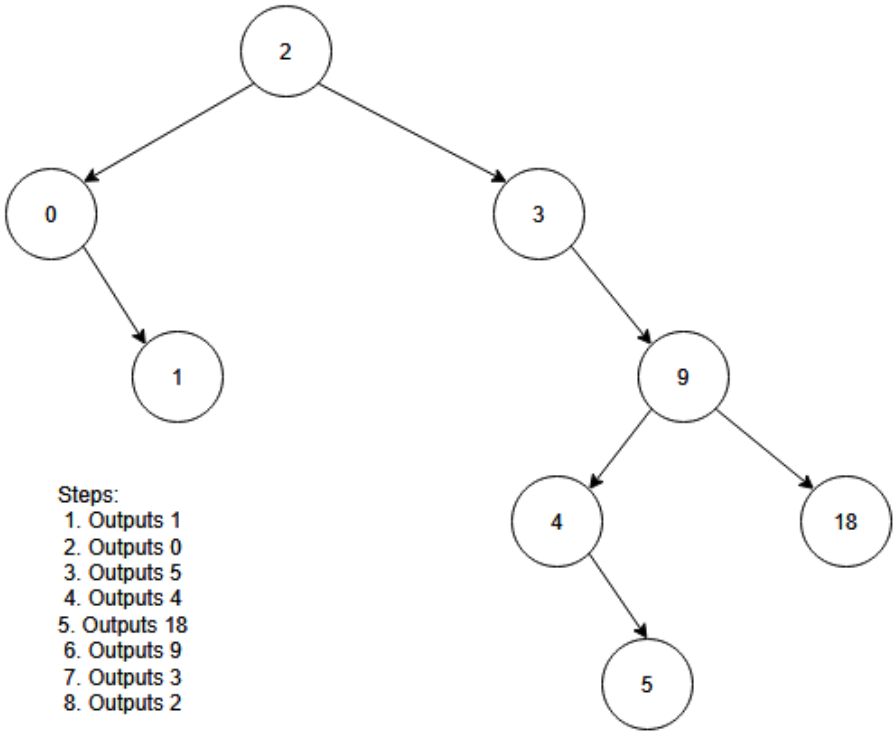


Figure 5: Post-Order Traversal Diagram

Observations of Figure 5:

- This diagram shows how leaf and child nodes are processed before moving upward to the parent nodes and ends at the root node.

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

```
template <typename T>
void BinarySearchTree<T>::preOrder() {  // From root -> left -> right -> repeat
    std::cout << key << " ";
    if (left != nullptr) {
        left -> preOrder();
    }

    if (right != nullptr) {
        right -> preOrder();
    }
}
```

Figure 6: Code Snippet of Pre-order Traversal

Observations of Figure 6:

- The ‘preOrder’ function process the current node’s key first before recursively calling itself on child pointers.

```
Pre Order: 2 0 1 3 9 4 5 18
```

Figure 7: Output Snippet of Pre-Order Traversal

Observations of Figure 7:

- This prints the expected output of the program that proves how a Pre-Order traversal works based on the shown diagram.

```
template <typename T>
void BinarySearchTree<T>::inOrder() { // From left -> root -> right -> repeat
    // Go to left
    if (left != nullptr) {
        left -> inOrder();
    }
    std::cout << key << " ";

    // Go to Right
    if (right != nullptr) {
        right -> inOrder();
    }
}
```

Figure 8: Code Snippet of In-order Traversal

Observations of Figure 8:

- The ‘inOrder’ function recursively visits the left child branch and displays the current node’s key and traverses to the right branch.

```
In Order: 0 1 2 3 4 5 9 18
```

Figure 9: Output Snippet of In-Order Traversal

Observations of Figure 9:

- This snippet displays all the values in an ascending order.

```
template <typename T>
void BinarySearchTree<T>::postOrder() { // From left -> right -> root -> repeat
    // Go to left
    if (left != nullptr) {
        left -> postOrder();
    }

    // Go to Right
    if (right != nullptr) {
        right -> postOrder();
    }
    std::cout << key << " ";
}
```

Figure 10: Code Snippet of Post-order Traversal

Observations of Figure 10:

- The 'postOrder' function recursively visits the left and right child pointers before printing the current node's key.



Figure 11: Output Snippet of Post-Order Traversal

Observations of Figure 11:

- This figure displays the postOrder step by step manual which ends at the root node of 2.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.